

Using uv in GitLab CI/CD

Using the uv image

Astral provides Docker images with uv preinstalled. Select a variant that is suitable for your workflow.

```
variables:
  UV_VERSION: "0.12.7"
  PYTHON_VERSION: "3.12"
  BASE_LAYER: trixie-slim
  # GitLab CI creates a separate mountpoint for the build directory,
  # so we need to copy instead of using hard links.
  UV_LINK_MODE: copy

uv:
  image: ghcr.io/astral-sh/uv:$UV_VERSION-python$PYTHON_VERSION-$BASE_LAYER
  script:
    # your `uv` commands
```

!!! note

If you are using a distroless image, you have to specify the entrypoint:

```
```yaml
uv:
 image:
 name: ghcr.io/astral-sh/uv:$UV_VERSION
 entrypoint: [""]
 # ...
```
```

Caching

Persisting the uv cache between workflow runs can improve performance.

```
uv-install:
  variables:
    UV_CACHE_DIR: .uv-cache
  cache:
    - key:
        files:
          - uv.lock
        paths:
          - $UV_CACHE_DIR
  script:
    # Your `uv` commands
  after_script:
    - uv cache prune --ci
```

See the GitLab caching documentation for more details on configuring caching.

Using `uv cache prune --ci` at the end of the job is recommended to reduce cache size. See the uv cache documentation for more details.

Using uv pip

If using the `uv pip` interface instead of the `uv project` interface, uv requires a virtual environment by default. To allow installing packages into the system environment, use the `--system` flag on all uv invocations or set the `UV_SYSTEM_PYTHON` variable.

The `UV_SYSTEM_PYTHON` variable can be defined in at different scopes. You can read more about how variables and their precedence works in GitLab [here](#)

Opt-in for the entire workflow by defining it at the top level:

```
variables:  
  UV_SYSTEM_PYTHON: 1
```

```
# [...]
```

To opt-out again, the `--no-system` flag can be used in any `uv` invocation.

When persisting the cache, you may want to use `requirements.txt` or `pyproject.toml` as your cache key files instead of `uv.lock`.